

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Объектно-ориентированное программирование»
Вариант 9

Выполнила:
Гинч Диана Исмаиловна
3 курс, группа ИВТ-б-о-23-1,
09.03.01 «Информатика и вычислительная
техника (профиль) «Программное
обеспечение средств вычислительной
техники и автоматизированных систем»,
очная форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и электроники,
Воронкин Р. А.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Тема: Элементы объектно-ориентированного программирования в языке Python

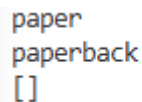
Цель работы: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Ссылка на репозиторий: [dianaasss/lab1: Лабораторная работа №1 по ООП](https://github.com/dianaasss/lab1)

Методика и порядок выполнения работы

1. Выполнили примеры

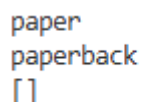
```
class Book:  
    material = 'paper'  
    cover = 'paperback'  
    all_books = []
```



```
paper  
paperback  
[]
```

Рисунок 1. Атрибут класса Book

```
class Book:  
    material = 'paper'  
    cover = 'paperback'  
    all_books = []  
  
# экземпляр класса Book  
my_book = Book()  
  
print(my_book.material)  
print(my_book.cover)  
print(my_book.all_books)
```



```
paper  
paperback  
[]
```

Рисунок 2. Экземпляр класса

```
class River:  
    # список всех рек  
    all_rivers = []
```

```

def __init__(self, name, length):
    self.name = name
    self.length = length
    # добавляем текущую реку в список всех рек
    River.all_rivers.append(self)

volga = River('Волга', 3530)
seine = River('Сена', 776)
nile = River('Нил', 6852)

# далее печатаем все названия рек
for river in River.all_rivers:
    print(river.name)

```

Волга
Сена
Нил

Рисунок 3. Использование метода `__init__`

```

class River:
    # список всех рек
    all_rivers = []

    def __init__(self, name, length):
        self.name = name
        self.length = length
        # добавляем текущую реку в список всех рек
        River.all_rivers.append(self)

    def get_info(self):
        print('Длина {0} равна {1} км'.format(self.name, self.length))

# Создаём реки
volga = River('Волга', 3530)
seine = River('Сена', 776)
nile = River('Нил', 6852)

# далее печатаем все названия рек
for river in River.all_rivers:
    print(river.name)

```

```
# Выводим информацию о реках
volga.get_info()
seine.get_info()
nile.get_info()

Волга
Сена
Нил
Длина Волга равна 3530 км
Длина Сена равна 776 км
Длина Нил равна 6852 км
```

Рисунок 4. Аргумент self

```
class Pet:
    kind = 'mammal'
    n_pets = 0 # количество питомцев
    pet_names = [] # список имен всех питомцев

    def __init__(self, spec, name):
        self.spec = spec
        self.name = name
        self.legs = 4

tom = Pet('cat', 'Tom')
avocado = Pet('dog', 'Avocado')
ben = Pet('goldfish', 'Benjamin')

# получим доступ к атрибуту класса напрямую через класс
Pet.n_pets += 3
print(Pet.n_pets)
print(tom.n_pets)
print(avocado.n_pets)
print(ben.n_pets)

3
3
3
3
PS C:\Users\diana>
```

Рисунок 5. Получение доступа к атрибуту класса напрямую через класс

```
ben.kind = 'fish'
print(Pet.kind)
print(tom.kind)
```

```

print(avocado.kind)
print(ben.kind)

tom.pet_names.append(tom.name)
avocado.pet_names.append(avocado.name)
ben.pet_names.append(ben.name)
print(Pet.pet_names)
print(tom.pet_names)
print(avocado.pet_names)
print(ben.pet_names)

tom.pet_names = ['Tom']
avocado.pet_names = ['Avocado']
ben.pet_names = ['Benjamin']
print(Pet.pet_names)
print(tom.pet_names)
print(avocado.pet_names)
print(ben.pet_names)
# изменение атрибута экземпляра
ben.legs = 0

Pet.all_specs = [tom.spec, avocado.spec, ben.spec]
print(tom.all_specs)
print(avocado.all_specs)
print(ben.all_specs)

avocado.breed = 'corgi'
mammal
mammal
mammal
fish
['Tom', 'Avocado', 'Benjamin']
['Tom', 'Avocado', 'Benjamin']
['Tom', 'Avocado', 'Benjamin']
['Tom', 'Avocado', 'Benjamin']
['Tom', 'Avocado', 'Benjamin']
['Tom']
['Avocado']
['Benjamin']
['cat', 'dog', 'goldfish']
['cat', 'dog', 'goldfish']
['cat', 'dog', 'goldfish']

```

Рисунок 6. Добавление и изменение атрибутов

```
class Ship:
```

```

def __init__(self, name, capacity):
    self.name = name
    self.capacity = capacity
    self.cargo = 0

def sail(self):
    print('{} has sailed!'.format(self.name))

def sail_function(name):
    print('{} has sailed!'.format(name))

def convert_cargo(self):
    cargo_kg = self.cargo * 1000
    return cargo_kg

# Создаём корабль
black_pearl = Ship('Black Pearl', 800)

# Используем методы
black_pearl.sail() # Black Pearl has sailed!
Ship.sail_function(black_pearl.name) # Black Pearl has sailed!

# Устанавливаем груз и конвертируем
black_pearl.cargo = 50 # добавляем груз
print(black_pearl.convert_cargo()) # 50000
    Black Pearl has sailed!
    Black Pearl has sailed!
    50000

```

Рисунок 7. Создание методы

```

class Ship:
    def __init__(self, name, capacity):
        self.name = name
        self.capacity = capacity
        self.cargo = 0

    def name_captain(self, cap):
        self.captain = cap
        print('{} is the captain of the {}'.format(self.captain,
self.name))

```

```

black_pearl = Ship('Black Pearl', 800)
black_pearl.name_captain('Jack Sparrow')
print(black_pearl.captain)
    Jack Sparrow is the captain of the Black Pearl
    Jack Sparrow
    - - - - -

```

Рисунок 8. Создание атрибутов с помощью методов

```

class Ship:
    def __init__(self, name, capacity):
        self.name = name
        self.capacity = capacity
        self.cargo = 0

    def load_cargo(self, weight):
        if self.cargo + weight <= self.capacity:
            self.cargo += weight
            print('Loaded {} tons'.format(weight))
        else:
            print('Cannot load that much')

    def unload_cargo(self, weight):
        if self.cargo - weight >= 0:
            self.cargo -= weight
            print('Unloaded {} tons'.format(weight))
        else:
            print('Cannot unload that much')

    # ДОБАВЬТЕ ЭТОТ МЕТОД В КЛАСС!
    def get_info(self):
        print("Ship: {}, Capacity: {} tons, Current cargo: {}
tons".format(
            self.name, self.capacity, self.cargo))

    # Только ПОСЛЕ этого создавайте объекты
ship1 = Ship("Titanic", 50000)
ship1.load_cargo(20000)
ship1.load_cargo(15000)
ship1.get_info() # Теперь этот метод существует!
ship1.unload_cargo(10000)

```

```

ship1.get_info()
    Loaded 20000 tons
    Loaded 15000 tons
    Ship: Titanic, Capacity: 50000 tons, Current cargo: 35000 tons
    Unloaded 10000 tons
    Ship: Titanic, Capacity: 50000 tons, Current cargo: 25000 tons

```

Рисунок 9. Изменение атрибутов с помощью методов

```

class Rectangle:
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    def get_width(self):
        return self.__width

    def set_width(self, w):
        self.__width = w

    def get_height(self):
        return self.__height

    def set_height(self, h):
        self.__height = h

    def area(self):
        return self.__width * self.__height

# Пример использования
rect = Rectangle(5, 3)

print("Ширина:", rect.get_width())    # 5
print("Высота:", rect.get_height())   # 3
print("Площадь:", rect.area())        # 15

# Изменяем размеры
rect.set_width(7)
rect.set_height(4)

print("\nПосле изменения:")
print("Ширина:", rect.get_width())    # 7

```



```

print("Высота:", rect.get_height()) # 4
print("Площадь:", rect.area())      # 28

        Ширина: 5
        Высота: 3
        Площадь: 15

        После изменения:
        Ширина: 7
        Высота: 4
        Площадь: 28

```

Рисунок 10. Уровни доступа к атрибутам

```

class Rectangle:
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    @property
    def width(self):
        return self.__width

    @width.setter
    def width(self, w):
        if w > 0:
            self.__width = w
        else:
            raise ValueError("Width must be positive")

    @property
    def height(self):
        return self.__height

    @height.setter
    def height(self, h):
        if h > 0:
            self.__height = h
        else:
            raise ValueError("Height must be positive")

    def area(self):
        return self.__width * self.__height

```

```

# Пример использования
rect = Rectangle(5, 3)

print("Ширина:", rect.width)    # 5 (работает геттер)
print("Высота:", rect.height)   # 3 (работает геттер)
print("Площадь:", rect.area())  # 15

# Изменяем размеры через сеттеры
rect.width = 7    # работает сеттер
rect.height = 4   # работает сеттер

print("\nПосле изменения:")
print("Ширина:", rect.width)    # 7
print("Высота:", rect.height)   # 4
print("Площадь:", rect.area())  # 28

# Попытка установить отрицательное значение
try:
    rect.width = -5 # вызовет ValueError
except ValueError as e:
    print("Ошибка:", e)

```

Ширина: 5
 Высота: 3
 Площадь: 15

После изменения:
 Ширина: 7
 Высота: 4
 Площадь: 28
 Ошибка: Width must be positive

Рисунок 11. Свойства

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

class Rational:
    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError("Denominator cannot be zero")

```

```

self.__numerator = abs(a)
self.__denominator = abs(b)

self.__reduce()

# Сокращение дроби
def __reduce(self):
    # Функция для нахождения наибольшего общего делителя
    def gcd(a, b):
        if a == 0:
            return b
        elif b == 0:
            return a
        elif a >= b:
            return gcd(a % b, b)
        else:
            return gcd(a, b % a)

    c = gcd(self.__numerator, self.__denominator)
    self.__numerator //= c
    self.__denominator //= c

@property
def numerator(self):
    return self.__numerator

@property
def denominator(self):
    return self.__denominator

# Прочитать значение дроби с клавиатуры. Дробь вводится как a/b.
def read(self, prompt=None):
    line = input() if prompt is None else input(prompt)
    parts = list(map(int, line.split('/', maxsplit=1)))

    if parts[1] == 0:
        raise ValueError("Denominator cannot be zero")

```

```

self.__numerator = abs(parts[0])
self.__denominator = abs(parts[1])
self.__reduce()

# Вывести дробь на экран
def display(self):
    print(f"{self.__numerator}/{self.__denominator}")

# Сложение обыкновенных дробей.
def add(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + \
            self.denominator * rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError("Argument must be Rational")

# Вычитание обыкновенных дробей.
def sub(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator - \
            self.denominator * rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError("Argument must be Rational")

# Умножение обыкновенных дробей.
def mul(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError("Argument must be Rational")

# Деление обыкновенных дробей.
def div(self, rhs):

```

```

    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator
        return Rational(a, b)
    else:
        raise ValueError("Argument must be Rational")

def equals(self, rhs):
    if isinstance(rhs, Rational):
        return (self.numerator == rhs.numerator) and \
            (self.denominator == rhs.denominator)
    else:
        return False

def greater(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator
        return v1 > v2
    else:
        return False

def less(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator
        return v1 < v2
    else:
        return False

if __name__ == '__main__':
    r1 = Rational(3, 4)
    r1.display()

    r2 = Rational()
    r2.read("Введите обыкновенную дробь: ")
    r2.display()

    r3 = r2.add(r1)

```

```

r3.display()

r4 = r2.sub(r1)
r4.display()

r5 = r2.mul(r1)
r5.display()

r6 = r2.div(r1)
r6.display()

```

$\frac{3}{4}$
 Введите обыкновенную дробь: $\frac{5}{6}$
 $\frac{5}{6}$
 $\frac{19}{12}$
 $\frac{1}{12}$
 $\frac{5}{8}$
 $\frac{10}{9}$

Рисунок 12. Класс Rational для работы с дробями

2. Выполнили индивидуальное задание 1

Парой называется класс с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать:

- метод инициализации `init`; метод должен контролировать значения аргументов на корректность;
- ввод с клавиатуры `read`;
- вывод на экран `display`.

Реализовать внешнюю функцию с именем `make_тип()`, где `тип` - тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

Номер варианта необходимо уточнить у преподавателя. В раздел программы, начинающийся после инструкции `if name == 'main':` добавить код, демонстрирующий возможности разработанного класса.

Вариант 9.

Поле first - целое положительное число, часы. Поле second - целое положительное число, минуты.

Реализовать метод minutes() - приведение времени в минуты.

Код программы:

```
class Time:
    def __init__(self, hours, mins):
        if not isinstance(hours, int) or hours <= 0:
            print("Ошибка: часы должны быть целым положительным числом")
            exit(1)
        if not isinstance(mins, int) or mins <= 0:
            print("Ошибка: минуты должны быть целым положительным числом")
            exit(1)
        self.hours = hours
        self.mins = mins

    def read(self):
        try:
            h = int(input("Введите часы (целое положительное число): "))
            m = int(input("Введите минуты (целое положительное число): "))
            if h <= 0 or m <= 0:
                print("Ошибка: введены не положительные числа")
                exit(1)
            self.hours = h
            self.mins = m
        except ValueError:
            print("Ошибка: введены некорректные данные")
            exit(1)

    def display(self):
        print(f"Время: {self.hours} часов {self.mins} минут")

    def minutes(self):
        return self.hours * 60 + self.mins

def make_Time(hh, mm):
    return Time(hh, mm)

import time
from my_time import Time, make_Time

if __name__ == "__main__":
    print("Создание объекта через ввод с клавиатуры:")
    my_time = Time(1, 1)
    my_time.read()
    my_time.display()
```

```
print("Всего минут:", my_time.minutes())
```

```
import pytest
from my_time import Time, make_Time

def test_creation_valid():
    time = Time(2, 30)
    assert time.hours == 2
    assert time.mins == 30
    assert time.minutes() == 150

def test_creation_invalid_hours():
    with pytest.raises(SystemExit):
        Time(-1, 30)

def test_creation_invalid_minutes():
    with pytest.raises(SystemExit):
        Time(2, -5)

def test_creation_zero_values():
    with pytest.raises(SystemExit):
        Time(0, 30)
    with pytest.raises(SystemExit):
        Time(2, 0)

def test_minutes_calculation():
    test_cases = [
        (1, 30, 90),
        (2, 0, 120),
        (0, 45, 45),
        (3, 15, 195)
    ]

    for hours, mins, expected_minutes in test_cases:
        if hours > 0 and mins > 0:
            time = Time(hours, mins)
            assert time.minutes() == expected_minutes

def test_display(capsys):
    time = Time(3, 45)
    time.display()

    captured = capsys.readouterr()
```



```

assert "Время: 3 часов 45 минут" in captured.out

def test_read_valid_input(monkeypatch):
    time = Time(1, 1) # создаем с временными значениями

    inputs = iter(["5", "20"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))

    time.read()

    assert time.hours == 5
    assert time.mins == 20
    assert time.minutes() == 320

def test_read_invalid_negative(monkeypatch):
    time = Time(1, 1)

    inputs = iter(["-5", "20"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))

    with pytest.raises(SystemExit):
        time.read()

def test_read_invalid_zero(monkeypatch):
    time = Time(1, 1)

    inputs = iter(["0", "30"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))

    with pytest.raises(SystemExit):
        time.read()

def test_read_invalid_type(monkeypatch):
    time = Time(1, 1)

    inputs = iter(["abc", "30"])
    monkeypatch.setattr('builtins.input', lambda _: next(inputs))

    with pytest.raises(SystemExit):
        time.read()

def test_make_Time_function():
    time = make_Time(4, 15)

    assert isinstance(time, Time)
    assert time.hours == 4

```

```

assert time.mins == 15
assert time.minutes() == 255

def test_make_Time_invalid():
    with pytest.raises(SystemExit):
        make_Time(-1, 30)

def test_edge_cases():

    time1 = Time(1, 1)
    assert time1.minutes() == 61

    time2 = Time(24, 59)
    assert time2.minutes() == 24 * 60 + 59

def test_consistency():
    time = Time(2, 45)

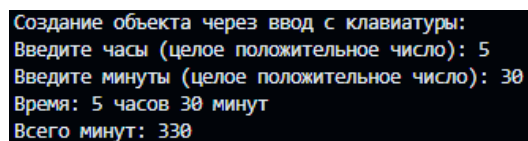
    expected_minutes = time.hours * 60 + time.mins
    assert time.minutes() == expected_minutes

def test_string_representation():
    time = Time(2, 30)

if __name__ == "__main__":

    pytest.main([__file__, "-v"])

```



```

Создание объекта через ввод с клавиатуры:
Введите часы (целое положительное число): 5
Введите минуты (целое положительное число): 30
Время: 5 часов 30 минут
Всего минут: 330

```

Рисунок 13. Результат работы программы

3. Выполнили индивидуальное задание 2

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы:

- метод инициализации `init`;
- ввод с клавиатуры `read`;
- вывод на экран `display`.

Номер варианта необходимо уточнить у преподавателя. В раздел программы, начинающийся после инструкции `if name == 'main':` добавить код, демонстрирующий возможности разработанного класса.

Реализовать класс `Account`, представляющий собой банковский счет. В классе должны быть четыре поля: фамилия владельца, номер счета, процент начисления и сумма в рублях. Открытие нового счета выполняется операцией инициализации. Необходимо выполнять следующие операции: сменить владельца счета, снять некоторую сумму денег со счета, положить деньги на счет, начислить проценты, перевести сумму в доллары, перевести сумму в евро, получить сумму прописью (преобразовать в числительное).

Код:

```
class Account:
    def __init__(self, owner='', account_number='', interest_rate=0.0,
balance=0.0):
        self.owner = owner
        self.account_number = account_number
        self.interest_rate = interest_rate
        self.balance = balance

    def read(self):
        self.owner = input("Введите фамилию владельца: ")
        self.account_number = input("Введите номер счета: ")
        self.interest_rate = float(input("Введите процент начисления: "))
        self.balance = float(input("Введите начальную сумму в рублях: "))

    def display(self):
        print(f"Владелец: {self.owner}")
        print(f"Номер счета: {self.account_number}")
        print(f"Процент: {self.interest_rate}%")
        print(f"Баланс: {self.balance:.2f} руб.")

    def change_owner(self, new_owner):
        self.owner = new_owner
        print(f"Владелец счета изменен на {self.owner}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Недостаточно средств для снятия.")
        else:
            self.balance -= amount
            print(f"Снято {amount:.2f} руб. Текущий баланс: {self.balance:.2f}
руб.")
```

```

def deposit(self, amount):
    self.balance += amount
    print(f"Внесено {amount:.2f} руб. Текущий баланс: {self.balance:.2f} руб.")

def accrue_interest(self):
    self.balance += self.balance * (self.interest_rate / 100)
    print(f"Начислены проценты: {self.interest_rate}%. Текущий баланс: {self.balance:.2f} руб.")

def to_dollars(self, rate=80.0):
    dollars = self.balance / rate
    print(f"Баланс в долларах: {dollars:.2f} $")
    return dollars

def to_euros(self, rate=90.0):
    euros = self.balance / rate
    print(f"Баланс в евро: {euros:.2f} €")
    return euros

def amount_in_words(self):
    words = self.number_to_words(int(self.balance))
    print(f"Баланс прописью: {words} рублей")
    return words

@staticmethod
def number_to_words(n):
    units = [
        "", "один", "два", "три", "четыре", "пять", "шесть", "семь", "восемь", "девять"
    ]
    tens = [
        "", "десять", "двадцать", "тридцать", "сорок", "пятьдесят", "шестьдесят", "семьдесят", "восемьдесят", "девяносто"
    ]
    teens = [
        "десять", "одиннадцать", "двенадцать", "тринадцать", "четырнадцать",
        "пятнадцать", "шестнадцать", "семнадцать", "восемнадцать", "девятнадцать"
    ]
    hundreds = [
        "", "сто", "двести", "триста", "четыреста", "пятьсот", "шестьсот", "семьсот", "восемьсот", "девятьсот"
    ]

    if n == 0:
        return "ноль"

    words = []

    if n >= 1000000:
        m = n // 1000000
        words.append(Account.number_to_words(m) + " миллион")
        n %= 1000000

    if n >= 1000:
        t = n // 1000

```

```

        if t == 1:
            words.append("одна тысяча")
        elif t == 2:
            words.append("две тысячи")
        else:
            words.append(Account.number_to_words(t) + " тысяч")
        n %= 1000

    if n >= 100:
        h = n // 100
        words.append(hundreds[h])
        n %= 100

    if 10 <= n <= 19:
        words.append(teens[n-10])
        n = 0

    if n >= 10:
        t = n // 10
        words.append(tens[t])
        n %= 10

    if n > 0:
        words.append(units[n])

    return ' '.join(words)

```

```

from my_account import Account

```

```

if __name__ == "__main__":
    account1 = Account()
    print("Создание первого счета:")
    account1.read()

    # Сначала снимаем сумму
    withdraw_amount = float(input("Введите сумму для снятия: "))
    account1.withdraw(withdraw_amount)

    # Потом вносим сумму
    deposit_amount = float(input("Введите сумму для внесения: "))
    account1.deposit(deposit_amount)

    # Начисляем проценты
    account1.accrue_interest()

    # Итоговый баланс
    print("\nИтоговый баланс первого счета:")
    account1.display()
    account1.to_dollars()
    account1.to_euros()

```

```

account1.amount_in_words()

# Замена владельца и повтор операций
print("\nЗамена владельца и повтор операций:")
new_owner = input("Введите нового владельца счета: ")
account1.change_owner(new_owner)

withdraw_amount = float(input("Введите сумму для снятия: "))
account1.withdraw(withdraw_amount)

deposit_amount = float(input("Введите сумму для внесения: "))
account1.deposit(deposit_amount)

account1.accrue_interest()

print("\nИтоговый баланс после смены владельца:")
account1.display()
account1.to_dollars()
account1.to_euros()
account1.amount_in_words()

```

```

import pytest
from io import StringIO
import sys
from my_account import Account

def test_account_creation_default():
    account = Account()
    assert account.owner == ''
    assert account.account_number == ''
    assert account.interest_rate == 0.0
    assert account.balance == 0.0

def test_account_creation_with_params():
    account = Account("Иванов", "1234567890", 5.0, 10000.0)
    assert account.owner == "Иванов"
    assert account.account_number == "1234567890"
    assert account.interest_rate == 5.0
    assert account.balance == 10000.0

def test_display(capsys):
    account = Account("Петров", "0987654321", 3.5, 5000.0)
    account.display()

    captured = capsys.readouterr()
    output = captured.out

```

```

assert "Владелец: Петров" in output
assert "Номер счета: 0987654321" in output
assert "Процент: 3.5%" in output
assert "Баланс: 5000.00 руб." in output

def test_change_owner():
    account = Account("Иванов", "123", 5.0, 1000.0)
    account.change_owner("Сидоров")

    assert account.owner == "Сидоров"

def test_deposit():
    account = Account(balance=1000.0)
    account.deposit(500.0)

    assert account.balance == 1500.0

def test_withdraw_success():
    account = Account(balance=1000.0)
    account.withdraw(300.0)

    assert account.balance == 700.0

def test_withdraw_insufficient_funds(capsys):
    account = Account(balance=100.0)
    account.withdraw(200.0)

    assert account.balance == 100.0

    captured = capsys.readouterr()
    assert "Недостаточно средств" in captured.out

def test_accrue_interest():
    account = Account(balance=1000.0, interest_rate=10.0)
    account.accrue_interest()

    assert account.balance == 1100.0 # 1000 + 10%

def test_to_dollars(capsys):
    account = Account(balance=8000.0)
    dollars = account.to_dollars(80.0)

    assert dollars == 100.0 # 8000 / 80 = 100

    captured = capsys.readouterr()

```

```

assert "Баланс в долларах:" in captured.out

def test_to_euros(capsys):
    account = Account(balance=9000.0)
    euros = account.to_euros(90.0)

    assert euros == 100.0 # 9000 / 90 = 100

    captured = capsys.readouterr()
    assert "Баланс в евро:" in captured.out

def test_number_to_words_static():
    assert Account.number_to_words(0) == "ноль"
    assert Account.number_to_words(1) == "один"
    assert Account.number_to_words(10) == "десять"
    assert Account.number_to_words(15) == "пятнадцать"
    assert Account.number_to_words(25) == "двадцать пять"
    assert Account.number_to_words(100) == "сто"
    assert Account.number_to_words(123) == "сто двадцать три"
    assert Account.number_to_words(1000) == "одна тысяча"
    assert Account.number_to_words(2000) == "две тысячи"
    assert Account.number_to_words(3500) == "три тысячи пятьсот"

def test_amount_in_words(capsys):
    """Тест вывода суммы прописью"""
    account = Account(balance=1234.56)
    words = account.amount_in_words()

    assert "тысяча" in words
    assert "двести" in words
    assert "тридцать" in words
    assert "четыре" in words

    captured = capsys.readouterr()
    assert "Баланс прописью:" in captured.out
    assert "рублей" in captured.out

def test_complex_operations():
    """Тест комплексных операций"""
    account = Account("Тестов", "111", 5.0, 2000.0)

    account.deposit(1000.0)
    assert account.balance == 3000.0

    account.withdraw(500.0)
    assert account.balance == 2500.0

```



```

account.accrue_interest()
assert account.balance == 2625.0

account.change_owner("НовыйВладелец")
assert account.owner == "НовыйВладелец"

def test_edge_cases():
    """Тест граничных случаев"""
    account = Account(balance=0.0)
    account.withdraw(100.0)
    assert account.balance == 0.0

    account = Account(balance=1000.0, interest_rate=-2.0)
    account.accrue_interest()
    assert account.balance == 980.0

def test_negative_deposit():
    """Тест внесения отрицательной суммы"""
    account = Account(balance=1000.0)
    account.deposit(-100.0)

if __name__ == "__main__":

    pytest.main([__file__, "-v"])

```

```

Создание первого счета:
Введите фамилию владельца: fgbhnmk
Введите номер счета: 598589598
Введите процент начисления: 25
Введите начальную сумму в рублях: 5000
Введите сумму для снятия: 500
Снято 500.00 руб. Текущий баланс: 4500.00 руб.
Введите сумму для внесения: 300
Внесено 300.00 руб. Текущий баланс: 4800.00 руб.
Начислены проценты: 25.0%. Текущий баланс: 6000.00 руб.

Итоговый баланс первого счета:
Владелец: fgbhnmk
Номер счета: 598589598
Процент: 25.0%
Баланс: 6000.00 руб.
Баланс в долларах: 75.00 $
Баланс в евро: 66.67 €
Баланс прописью: шесть тысяч рублей

Замена владельца и повтор операций:
Введите нового владельца счета: fcvghnmk
Владелец счета изменен на fcvghnmk
Введите сумму для снятия: 600
Снято 600.00 руб. Текущий баланс: 5400.00 руб.
Введите сумму для внесения: 500
Внесено 500.00 руб. Текущий баланс: 5900.00 руб.
Начислены проценты: 25.0%. Текущий баланс: 7375.00 руб.

Итоговый баланс после смены владельца:
Владелец: fcvghnmk
Номер счета: 598589598
Процент: 25.0%
Баланс: 7375.00 руб.
Баланс в долларах: 92.19 $
Баланс в евро: 81.94 €
Баланс прописью: семь тысяч триста семьдесят пять рублей

```

Рисунок 14. Результат выполнения программы

Контрольные вопросы

1. Как осуществляется объявление класса в языке Python? Объявление класса осуществляется с помощью ключевого слова `class`, за которым следует имя класса (обычно с заглавной буквы) и двоеточие. Тело класса пишется с отступом. `class MyClass: # атрибуты и методы класса pass`

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

1) Атрибуты класса принадлежат самому классу. Все экземпляры этого класса разделяют один и тот же атрибут класса. Изменение атрибута класса затронет все экземпляры.

2) Атрибуты экземпляра принадлежат конкретному объекту (экземпляру) класса. Они уникальны для каждого экземпляра. Изменение атрибута одного экземпляра не затронет другие.

3. Каково назначение методов класса? Методы класса определяют поведение объектов этого класса. Они являются функциями, которые описывают, что объекты класса могут делать. Методы могут работать как с атрибутами экземпляра, так и с атрибутами класса.

4. Для чего предназначен метод `__init__()` класса? Метод `__init__()` - это конструктор экземпляра класса. Он автоматически вызывается при создании нового объекта (экземпляра) класса. Его основное назначение - инициализировать атрибуты нового объекта, то есть задать ему начальное состояние.

5. Каково назначение `self`? `self` - это ссылка на текущий экземпляр класса. Через `self` методы получают доступ к атрибутам и другим методам этого конкретного объекта. Это первый параметр любого метода экземпляра, хотя сам язык не запрещает назвать его иначе, но это сильно противоречит общепринятым соглашениям.

6. Как добавить атрибуты в класс? 1) Атрибуты класса: объявляются прямо внутри тела класса, но вне любых методов. `class MyClass: class_attr = "Я атрибут класса" # Добавление атрибута класса` 2) Атрибуты экземпляра: обычно добавляются и инициализируются в методе `__init__` через `self`. `class`

MyClass: def __init__(self, value): self.instance_attr = value # Добавление атрибута экземпляра Также атрибут экземпляру можно добавить динамически в любом другом методе или даже вне класса, обратившись к объекту: obj = MyClass() obj.new_attr = 10

7. Как осуществляется управление доступом к методам и атрибутам в языке Python? Python использует модель «согласованного доступа» (we are all consenting adults here), а не строгое управление доступом. Для обозначения уровня доступа используются соглашения об именовании: 1) Публичные (public) атрибуты: attr_name 2) Защищенные (protected) атрибуты (сигнал для программиста, что с ним нужно быть осторожнее, а не для интерпретатора): _attr_name 3) Приватные (private) атрибуты (интерпретатор искажает их имя, чтобы усложнить случайный доступ извне, но прямой доступ все равно возможен): __attr_name

8. Каково назначение функции isinstance? Функция isinstance(object, classinfo) проверяет, является ли объект object экземпляром класса classinfo или экземпляром любого из его подклассов (дочерних классов). Возвращает True или False. Она используется для проверки типа объекта.

Вывод: приобрели навыки по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.